

Opis projekta

Cilj projekta je razvoj robustnog distribuiranog sistema za prikupljanje, obradu i čuvanje podataka dobijenih od senzorskih čvorova.

Sistem treba da obezbedi:

- 1 praćenje trenutnih vrednosti, pristup istorijskim vrednostima i evidentiranje događaja
- 2 toleranciju na otkaze
- 3 konzistentnost
- 4 pouzdanu i bezbednu klijent-server komunikaciju

Primer primene sistema je nadzor temperature u kritičnom industrijskom sistemu, na primer u jezgru nuklearne elektrane.

Sistem raspolaže većim brojem senzora postavljenih na različitim lokacijama, pri čemu u svakom trenutku tačno pet senzora treba da bude aktivno i da učestvuje u praćenju temperature.

1. Praćenje vrednosti

- Pre pokretanja simulacije svaki senzor dobija jedinstveni *ID*, opseg za generisanje temperature, kvalitet podataka (*GOOD*, *BAD* ili *UNCERTAIN*) i granice za aktiviranje alarma. Alarmi imaju prioritete 1, 2 i 3.
- Trenutne vrednosti se prate ispisom izmerene vrednosti i odgovarajućeg vremenskog trenutka na konzoli.
- Alarm registruje sam senzor kada izmerena vrednost pređe zadatu graničnu vrednost. Ako se aktivira alarm prioriteta 3, alarmi prioriteta 1 i 2 ne treba dodatno da se aktiviraju.
- U zavisnosti od prioriteta alarma, trenutnu vrednost treba ispisati na konzoli odgovarajućom bojom: žutom za prioritet 1, narandžastom za prioritet 2 i crvenom za prioritet 3. Nakon toga senzor šalje poruku serveru.
- Server ispisuje informaciju o senzoru na kom se alarm javio i vrednost koja je izazvala alarm, i to u boji koja odgovara prioritetu alarma.
- Za vrednosti kod kojih nije aktiviran alarm koristiti prioritet 0, radi jednostavnijeg zapisa podataka u bazu.
- Sve podatke zapisivati u bazu na serveru.

2. Sistem tolerantan na otkaze

- Sistem treba da obezbedi da u svakom trenutku postoji tačno 5 aktivnih senzora.
- Senzor se smatra neaktivnim ako server u periodu od 10 sekundi ne primi njegovu poruku.
- Za potrebe testiranja potrebno je omogućiti privremeno blokiranje senzora u trajanju od 30 sekundi.
- Server treba da vodi evidenciju o vremenskom trenutku poslednje primljene poruke za svaki senzor koji je ikada bio deo sistema.

3. Konzistentnost

- Aktivni senzori na svakih 1-10 sekundi mere temperaturu (generišu nasumičnu vrednost) i šalju je serveru.
- Primljene vrednosti se, korišćenjem *Entity Framework*-a, upisuju u PostgreSQL ili SQL Server bazu na serveru.
- Na svakih minut izračunava se konsenzus vrednost na osnovu podataka iz prethodnog minuta i upisuje se u bazu.
- Svaka zapisana vrednost treba da ima oznaku, odnosno *flag*, koji označava da li predstavlja konsenzus vrednost.
- Pretpostaviti da pojedini senzori mogu biti maliciozni: mogu prestati da odgovaraju, kasniti sa odgovorom ili slati neispravne podatke.
- Konsenzus vrednost treba izračunati primenom odabranog konsenzus algoritma. Potrebno je istražiti BFT pristup i implementirati pojednostavljenu verziju izabranog algoritma.
- Kada se utvrdi da je senzor maliciozan, njegov kvalitet podataka postavlja se na *BAD*. U izračunavanje konsenzus vrednosti ulaze samo podaci kvaliteta *GOOD*.

4. Pouzdana komunikacija

- Sve poruke koje klijent šalje serveru treba da budu šifrovane i digitalno potpisane, kako bi se obezbedila poverljivost sadržaja poruke i potvrdio identitet pošiljaoca.
Preporuka: AES i RSA/ECDSA.
- Za zaštitu od *replay* napada, klijent uz svaku poruku šalje vremenski trenutak slanja i jedinstveni *ID* poruke, koji se uvećava nakon svake poslate poruke.
- Sistem mora biti otporan na DoS napade od strane malicioznih senzora. Ako isti *ID* senzora pošalje više od 10 poruka u sekundi, server ga privremeno blokira.
Predlog: *AspNetCoreRateLimit*.
- Poruke ne treba razmenjivati isključivo preko *localhost* adrese, već preko konkretne mrežne adrese. Potrebno je analizirati bezbednosne rizike takvog pristupa, primeniti odgovarajuće mere zaštite i dokumentovati način na koji je bezbedna komunikacija ostvarena.
- Za komunikaciju između klijenata i servera koristiti HTTP/REST.

Sistem se sastoji od sledećih mikroservisa / komponenti:

- 1 *IngestionService* - servis za prijem podataka od senzora.
Njegova uloga je da primi podatke i prosledi ih na dalju obradu; treba da bude brz i skalabilan da bi mogao da obradi veliki broj pristiglih poruka.
- 2 *ConsensusService* - worker servis koji čita sirove podatke iz baze, računa konsenzus vrednost na svakih minut i upisuje rezultat u posebnu tabelu. Pristup je zasnovan na *Command Query Responsibility Segregation* obrascu, čime se obrada podataka odvaja od API-ja koji mora ostati dostupan klijentima.
- 3 *NotificationService* - servis zadužen za praćenje alarma.
IngestionService obaveštava ovaj servis kada detektuje alarm. Servis koristi SignalR za slanje obaveštenja klijentima u realnom vremenu.
- 4 *Ingress* - jedinstvena ulazna tačka sistema koja rutira saobraćaj ka odgovarajućim servisima, npr. */api/ingest* ka servisu za prijem podataka i */api/reports* ka servisu za izveštaje.

Važne napomene

- Za izradu projekta koristiti ASP.NET Core, Docker i Kubernetes (Minikube), uz lokalno pokretanje sistema pomoću *docker-compose* okruženja.
- GitHub repozitorijum treba da sadrži izvorni kod, *.yaml* konfiguracione fajlove, uputstvo za pokretanje sistema, opis primenjenih bezbednosnih mera i slike pokrenutog sistema.
- Projekat nosi 30 poena ukoliko se odbrani do 19. jula. Nakon tog roka moguće je ostvariti najviše 20 poena.
- Odbrane projekata do kraja semestra planirane su u redovnim terminima vežbi: 16/17. jun i 23/24. jun. Nakon toga, odbrane će biti organizovane po dogovoru.
- Tim čine najviše tri studenta.
- Na odbrani je potrebno demonstrirati rad sistema na najmanje dva računara, tako da budu pokrenuti serveri ili klijenti. Voditi računa o raspoloživosti i kapacitetima računara pri formiranju radnih grupa.